

Contents

Audio Classifier & Profanity Filter	1
Overview	1
Project Structure	1
File Reference	1
Setup	6
Training Data: MUSAN	6
Usage	7

Audio Classifier & Profanity Filter

Overview

This application analyzes audio input — either an uploaded file or a live microphone feed — to determine its content type (Speech, Music, or Noise), detect the spoken language, and identify profane content. It is built with **Flet** for the UI layer and **Python** for the audio-processing backend.

The project consists of two functional tracks:

1. **File-upload pipeline** — accepts an audio file, classifies it, detects language, and (on request) checks it for profanity.
2. **Live filter** — processes a microphone feed in real time and plays it back through the speaker with profane words bleeped out, using a short broadcast delay. This module is designed for eventual deployment as an **Auracast BIS sink** on Samsung devices, and its input/output layer is isolated accordingly to support that transition.

Project Structure

```
audio_app/
├── main.py                # Application entry point and UI (Flet)
├── audio_pipeline.py      # File-upload pipeline: classification, language
│                          # detection, transcription, profanity check
├── models/               # Trained model files (svm_model.pkl, scaler.pkl)
│                          # – not included
├── stagel/               # SVM training artifacts: extracted features,
│                          # evaluation results, confusion matrix
├── profanity/            # Profanity detection (wordlist + ML classifier)
├── speech/               # Standalone language-detection test utility
├── realtime/            # Live microphone-to-speaker filtering engine
└── requirements.txt
```

File Reference

main.py

The application’s entry point and the entirety of its UI, built with Flet. The file is organized into three parts:

- **Design system** — a `Palette` class defining the light and dark color schemes, and a set of reusable component builders (`card()`, `chip()`, `pill_button()`, `ring()`, `status_pill()`, etc.) used consistently across every screen.

- **Screen builders** — one function per screen: `build_home`, `build_result`, `build_history`, `build_live`, `build_analytics`, `build_settings`, `build_about`. Each returns the Flet component tree for that screen.
- **main(page)** — initializes the page theme and file picker, and defines a `navigate(screen_name)` router that switches the active screen and updates the bottom navigation bar. The module-level `ft.run(main)` call is the application's actual entry point.

Screen summary:

Screen	Function
Home	File picker (wav, mp3, ogg, flac, m4a, aac); a profanity-detection toggle set before a file is selected; a shortcut to the Live Filter screen; a "Recent" list from analysis history. File selection runs <code>analyze_audio()</code> on a background thread and navigates to Result.
Result	Displays detected type, language, duration, and timestamp, with an in-app audio player. If profanity detection is enabled and the clip is classified as Speech, automatically runs translation and profanity checking in the background and displays a category-by-category score breakdown.
History	Chronological list of past analyses (type, language, timestamp).
Live Filter	Start/stop control for the real-time engine, live status indicator, and a stats panel (detected language, seconds played/beeped, buffer depth, analysis pass count, unverified-segment count, input overflows, output underruns, active backend, delay, cushion, and active speaker/monitor).
Analytics	Aggregate scan count, per-type distribution (Speech/Music/Noise), and a language-frequency breakdown of past speech clips.
Settings	Theme toggle, history export (to .txt), and history clearing (with confirmation).
About	Version information and specification summaries for the SVM classifier and the Whisper model.

audio_pipeline.py

The file-upload analysis pipeline invoked from the Home screen. Loads three models once at import time: the SVM classifier and its feature scaler (from `models/`), and Whisper (`small`).

Function	Purpose
<code>extract_features(audio, sr)</code>	Extracts the 57 features consumed by the SVM classifier: 20 MFCC means, 20 MFCC standard deviations, 12 chroma means, and one value each for spectral centroid, spectral bandwidth, spectral rolloff, zero-crossing rate, and RMS. This is the exact feature set <code>stage1/Features.csv</code> was built from.
<code>classify_audio(path)</code>	Loads up to 10 seconds of audio at 16 kHz, extracts the 57 features, scales them, and returns "Speech", "Music", or "Noise".
<code>detect_language(path)</code>	Runs Whisper's language detection on the initial audio window and maps the result to one of 11 supported languages.
<code>transcribe_audio(path)</code>	Transcribes audio in its original (source) language.
<code>translate_to_english(path) /</code> <code>translate_array_to_english(array, sr)</code>	Translates audio to English using Whisper's translate task. The array variant operates on in-memory audio (used by the real-time pipeline) and expects 16 kHz input.
<code>check_profanity(text)</code>	Passes English text to the profanity package's ML detector and returns its verdict, with a safe fallback if the model failed to load.
<code>analyze_audio(path)</code>	The primary entry point called by <code>main.py</code> on every file upload. Runs classification, and language detection only when the type is Speech or Music. Transcription, translation, and profanity checking are intentionally deferred and triggered on demand from the Result screen, keeping the initial upload response fast.

Note: `audio_pipeline.py` performs an unconditional `import whisper` at module load, with no fallback. Without `openai-whisper` installed, this import fails — and since `main.py` imports `audio_pipeline` at startup, the application fails to launch as a result.

`models/`

Not source code — the expected location for two pre-trained artifacts:

- `models/svm_model.pkl`
- `models/scaler.pkl`

These must be supplied; they are not included in this repository. If retraining is required, see Training Data: MUSAN below.

`stage1/`

Reference artifacts documenting how the SVM classifier was trained. Not loaded by the application at runtime.

File	Contents
Features.csv	The MUSAN dataset after feature extraction: one row per clip, 57 feature columns (MFCC mean/standard deviation, chroma, spectral centroid/bandwidth/rolloff, zero-crossing rate, RMS) plus class label. This is the exact table <code>svm_model.pkl</code> and <code>scaler.pkl</code> were fit on. Training and evaluation results for the Speech/Music/Noise classifier. Classification results on the held-out test set, showing where the trained model confused one class for another.
Model_results.csv	
Confusion matrix	

These three artifacts together document classifier performance and provide a baseline for any future retraining or feature-set changes.

profanity/

Two complementary detection mechanisms:

File	Role
profanity_wordlist.py	Wordlist class providing fast, exact word/phrase matching with leetspeak normalization (<code>sh1t</code> → <code>shit</code>) and repeated-character collapsing (<code>fuuuck</code> → <code>fuuck</code>). Matching is whole-token by default to avoid false positives on innocuous words containing a flagged substring; a small set of unambiguous terms is also matched as a substring. Ships with a compact built-in English list; production deployments can supply additional per-language <code>.txt</code> files via <code>--wordlist-dir</code> (compatible with the community LDNOOBW list format). Because Whisper provides word-level timestamps, a wordlist match identifies the exact time range to bleep.
profanity_detector.py	ML-based detection using <code>unitary/multilingual-toxic-xlm-roberta</code> via <code>transformers.pipeline()</code> , loaded at import time. <code>detect_profanity(text)</code> scores text against six categories (<code>toxic</code> , <code>obscene</code> , <code>insult</code> , <code>threat</code> , <code>identity_attack</code> , <code>sexual_explicit</code>) and flags content as profane above a 70% confidence threshold, returning the top category, its score, and the full label breakdown. <code>score_texts(list)</code> is a batched variant used by the real-time filter to localize a sentence-level flag down to the specific word responsible.

File	Role
<code>__init__.py</code>	Lazily imports <code>detect_profanity / score_texts</code> so that importing the package alone does not load the transformer model unless it is actually invoked.
<code>test_profanity.py</code>	Manual test utility. Run without arguments for interactive text-mode testing, or supply an audio file path to run the complete pipeline (classification, transcription, translation, profanity check).

speech/

`speech_language_detection.py` — a standalone diagnostic script, independent of `main.py`. Loads `Whisper small` and prompts for an audio file path in a loop, printing the detected language for each. Intended for isolated testing of language detection.

realtime/

The live microphone-to-speaker filtering engine. This directory includes its own detailed `realtime/README.md`, covering CLI arguments, latency tuning, and the Auracast integration path in full; consult it before modifying this code.

Summary of contents:

File	Role
<code>realtime_filter.py</code>	Orchestrator. Builds the analyzer and runs the threaded live engine (capture → voice activity detection → transcription → profanity check → delayed, beeped playback); also provides an offline mode for file-in/file-out processing.
<code>config.py</code>	<code>RealtimeConfig</code> dataclass defining all tunable parameters (timing, delay, VAD mode, beep settings, profanity thresholds, backpressure limits), plus the CLI argument parser that constructs one.
<code>transcription.py</code>	A single <code>TranscriptionEngine</code> interface over three interchangeable backends: <code>faster-whisper</code> (recommended; approximately 4x faster on CPU), <code>openai-whisper</code> (fallback), and a dependency-free mock backend used for testing. All backends return an identical <code>Word(text, start, end)</code> structure.
<code>vad.py</code>	Voice activity detection, used to avoid running transcription on non-speech audio. <code>EnergyVAD</code> (default) is a dependency-free adaptive noise-floor gate; <code>SileroVAD</code> is an optional neural alternative for noisy or music-bed audio.

File	Role
audio_io.py	Input sources (MicSource, WavFileSource, StdinPCMSource), the SpeakerSink output, beep-tone generation and overlay, WAV read/write, streaming resampling, and buffer management.
selftest.py	Logic-level tests (resampling, chunk assembly, wordlist matching, beep placement, VAD) using the mock backend; requires no Whisper/torch installation.
livetest.py	End-to-end test of the threaded live engine using a simulated clock, verifying zero sample loss and correct beep placement.

The Auracast integration point is isolated to `audio_io.py`: adding a new input source class alongside `MicSource` and registering it in `make_source()` is sufficient to integrate a new transport without modifying the rest of the pipeline.

Setup

Python version: 3.10–3.12 is recommended, matching the versions `faster-whisper` and `torch` provide prebuilt wheels for.

System dependencies (not installable via `pip`):

```
\FunctionTok{sudo}\NormalTok{ apt install libportaudio2 } \CommentTok{ \# Required for  
↪ microphone/speaker I/O (sounddevice) }  
\FunctionTok{sudo}\NormalTok{ apt install ffmpeg } \CommentTok{ \# Required by Whisper to decode  
↪ audio files }
```

- On macOS: `brew install portaudio ffmpeg`
- On Windows: ensure `ffmpeg` is on the system `PATH`; `PortAudio` ships bundled with the `sound-device` wheel.

Python dependencies:

```
\ExtensionTok{pip}\NormalTok{ install } \AttributeTok{ {-}r } \NormalTok{ requirements.txt }
```

This installs the UI framework, the classification and Whisper stack, the profanity detection models, and the live audio I/O dependencies. See `requirements.txt` for the complete list.

Model files:

`audio_pipeline.py` requires two pre-trained files not included in this repository:

- `models/svm_model.pkl`
- `models/scaler.pkl`

Place existing trained files in this location, or retrain following the instructions below.

Once configured:

```
\ExtensionTok{python}\NormalTok{ main.py }
```

Training Data: MUSAN

The SVM classifier (Speech/Music/Noise; 57 features — MFCC, chroma, and spectral features; RBF kernel) was trained on **MUSAN**, a public corpus of music, speech, and noise recordings distributed by OpenSLR under a Creative Commons / public-domain license.

Download:

```
\FunctionTok{wget}\NormalTok{ https://www.openslr.org/resources/17/musan.tar.gz}
\FunctionTok{tar} \AttributeTok{{-}xvzf}\NormalTok{ musan.tar.gz}
```

This produces a musan/ directory containing three subfolders — music/, speech/, noise/ — each holding 16 kHz WAV files organized by source. This is the raw input expected by `extract_features()` in `audio_pipeline.py` for retraining `svm_model.pkl` and `scaler.pkl`. To reuse the already-extracted feature set instead of repeating extraction, refer to `stage1/Features.csv`.

This dataset is only required for retraining the classifier; it is not needed if `svm_model.pkl` and `scaler.pkl` are already available.

Usage

```
\ExtensionTok{python}\NormalTok{ main.py                                }\CommentTok{\# Launch the}
↪ application}
\ExtensionTok{python}\NormalTok{ realtime/realtime\_filter.py              }\CommentTok{\# Live filter,}
↪ standalone (microphone to speaker)}
\ExtensionTok{python}\NormalTok{ realtime/selftest.py                      }\CommentTok{\# Realtime engine}
↪ logic tests (no heavy dependencies)}
\ExtensionTok{python}\NormalTok{ realtime/livetest.py                      }\CommentTok{\# End{-}to{-}end}
↪ live{-}engine test with a simulated clock}
\ExtensionTok{python}\NormalTok{ profanity/test\_profanity.py              }\CommentTok{\# Profanity}
↪ detector – text or audio{-}file mode}
\ExtensionTok{python}\NormalTok{ speech/speech\_language\_detection.py    }\CommentTok{\# Standalone}
↪ Whisper language{-}detection utility}
```